

Week 2 : part 2. (partial) solutions for the exercise slide.

- Fast Selection [GT 9.2] [CLRS 9.3]

Exercise: Implement $\text{SPLIT}(S, x)$ in-place, i.e., by only swapping elements inside the array S .

Solution See CLRS 7.1 PARTITION

- Black-box Minimization: https://en.wikipedia.org/wiki/Golden-section_search

Exercise: Implement it / write out the pseudocode

Solution: see wikipedia for several implementations.

- Convex Hulls: with different algorithms [GT 22.2] [CLRS 33.3]

Exercise: Fill in the details of the other cases.

Solution: Just try it out on your own ;)

- Closest-Pair of Points: with different algorithms [22.4]

Exercise: Work through the analysis slide.

Solutions: (implementation as in 3. is omitted, but the timing analysis can still be done based on the overview.)

1. Suppose 5 or more points in L are found in a square of size $d \times d$. Divide the square into 4 smaller squares of size $\frac{d}{2} \times \frac{d}{2}$. At least one pair of points must fall within the same smaller square: these two points will then be at distance at most $\frac{d}{\sqrt{2}} < d$, which contradicts the assumption that every pair of points in L is at distance at least d .
2. The proof is by induction on the number of points. The algorithm is trivially correct for two points, so we may turn to the inductive step. Suppose we have n points and let (p_s, p_t) be the closest pair. There are three cases.

If $p_s, p_t \in L$, then $(p_s, p_t) = (p_L, q_L)$ by the inductive hypothesis and all the other pairs tested by the algorithm are at a larger distance apart, so the algorithm will correctly output (p_s, p_t) . The same reasoning holds if $p_s, p_t \in R$.

If $p_s \in L$ and $p_t \in R$, the algorithm will be correct as long as it tests the distance between p_s and p_t . Because p_s and p_t are at distance smaller than d , they will belong to the strip of points with x -coordinate in $[x-d, x+d]$. Suppose that $y_s \leq y_t$. A symmetric construction applies in the other case. Consider the rectangle S with vertices $(x-d, y_s), (x-d, y_s+d), (x+d, y_s+d), (x+d, y_s)$. Notice that both p_s and p_t must be contained in S . Moreover, the intersection of S with L is a square of size $d \times d$, which, by a), can contain at most 4 points, including p_s . Similarly, the intersection of S with R can also contain at most 4 points, including p_t . Because the algorithm checks the distance between p_s and the 7 points following p_s in the y -sorted list of points in the middle strip, it will check the distance between p_s and all the points of S . In particular, it will check the distance between p_s and p_t , as required for the correctness of the algorithm.

3 & 4.

When called on input of n points this algorithm first computes the median x value in $O(n)$ and then splits the list of points into those belonging to L and R , which also takes time $O(n)$. Then the algorithm can recurse on these two subproblems, each over $n/2$ points. Once these have been solved the algorithm sorts the points in the middle strip by y coordinate, which takes time $O(n \log n)$ and then computes $O(n)$ distances, each of which can be calculated in constant time. Hence the running time is given by the recursion $T(n) = 2T(\frac{n}{2}) + O(n \log n)$. This can be analyzed as in the proof of the Master theorem. The k th level of the recursion tree will contribute

$t_k = 2^k \frac{n}{2^k} (\log n - k)$. Hence, the total running time will be:

$$\sum_{k=0}^{\log n} t_k = n \log^2 n - n \sum_{k=0}^{\log n} k \leq n \log^2 n - \frac{n}{2} \log^2 n = O(n \log^2 n)$$

(Alternatively, prove that there are $c, n_0 > 0$ such that $T(n) \leq c(n \log^2 n)$ for all $n \geq n_0$ by induction on n .)

5. We can save some time by sorting the points by y -coordinate only once and making sure that the split routine is implemented as not to modify the order by y when splitting by x . Sorting takes time $O(n \log n)$, while the time required by the remaining of the algorithm is now described by the recurrence $T(n) = 2T(\frac{n}{2}) + O(n)$, which yields $T(n) = O(n \log n)$. Hence, the overall running time is $O(n \log n)$.